

Modulo_2: Algoritmos y Benchmarking

Pedro Farjo

2026-04-01

R Markdown

Este es el informe técnico para el Módulo 2, donde analizamos la eficiencia algorítmica y el uso de R-Cran.

Primeros comandos de R

R es un lenguaje muy parecido a Matlab y permite el trabajo con matrices[cite: 1].

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

Uso de datasets internos de R

Usando el comando `data` accedemos a tablas. Podemos elegir columnas específicas usando el signo `$`[cite: 1].

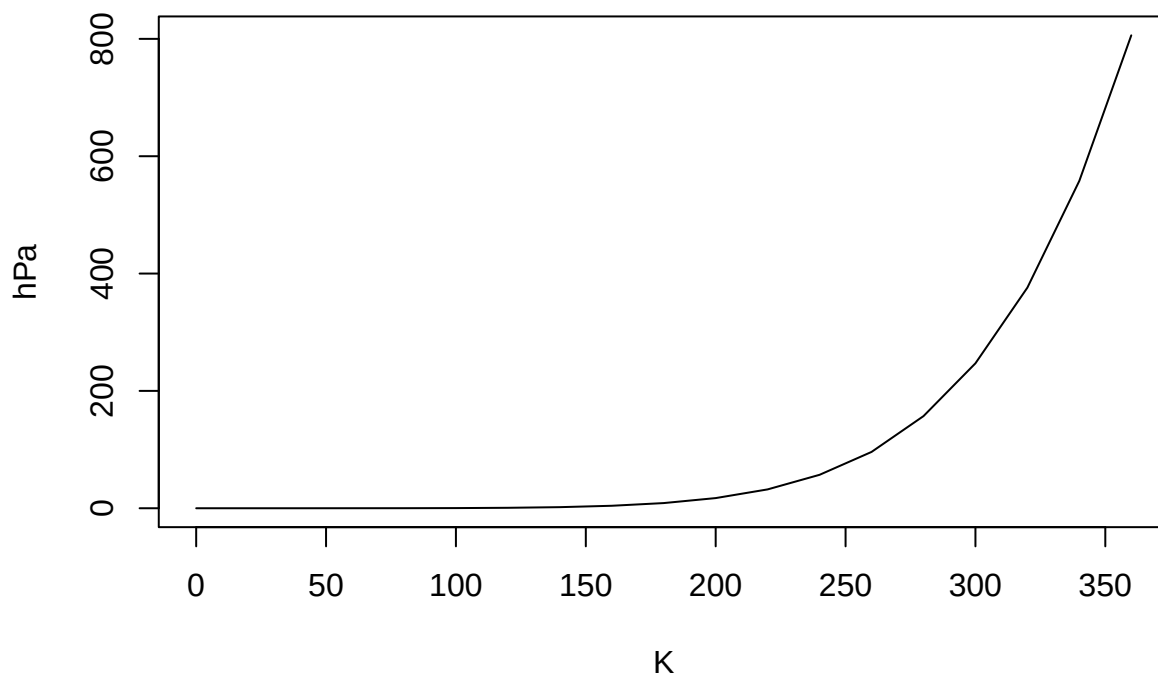
```
summary(cars)
```

```
##      speed          dist
##  Min.   : 4.0      Min.   :  2.00
## 1st Qu.:12.0      1st Qu.: 26.00
##  Median :15.0      Median : 36.00
##   Mean  :15.4      Mean    : 42.98
## 3rd Qu.:19.0      3rd Qu.: 56.00
##   Max.  :25.0      Max.     :120.00
```

Comando Plot

Nos permite graficar datos y asignar nombres a variables[cite: 1].

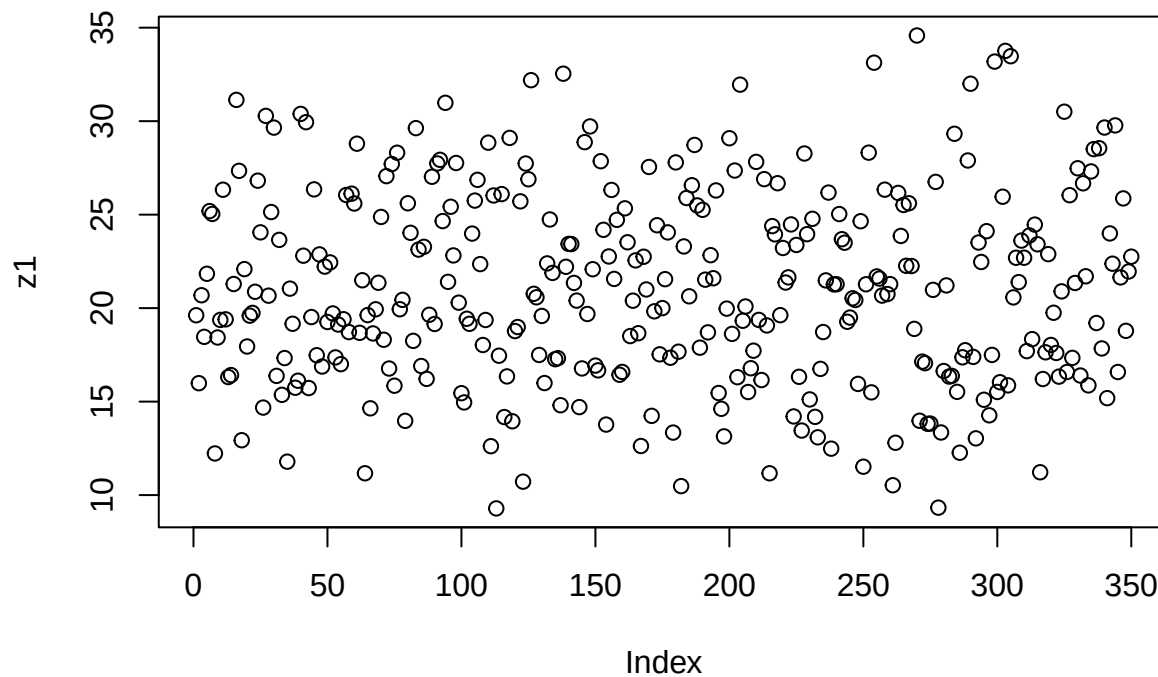
Presion del gas ideal



Funciones estadísticas

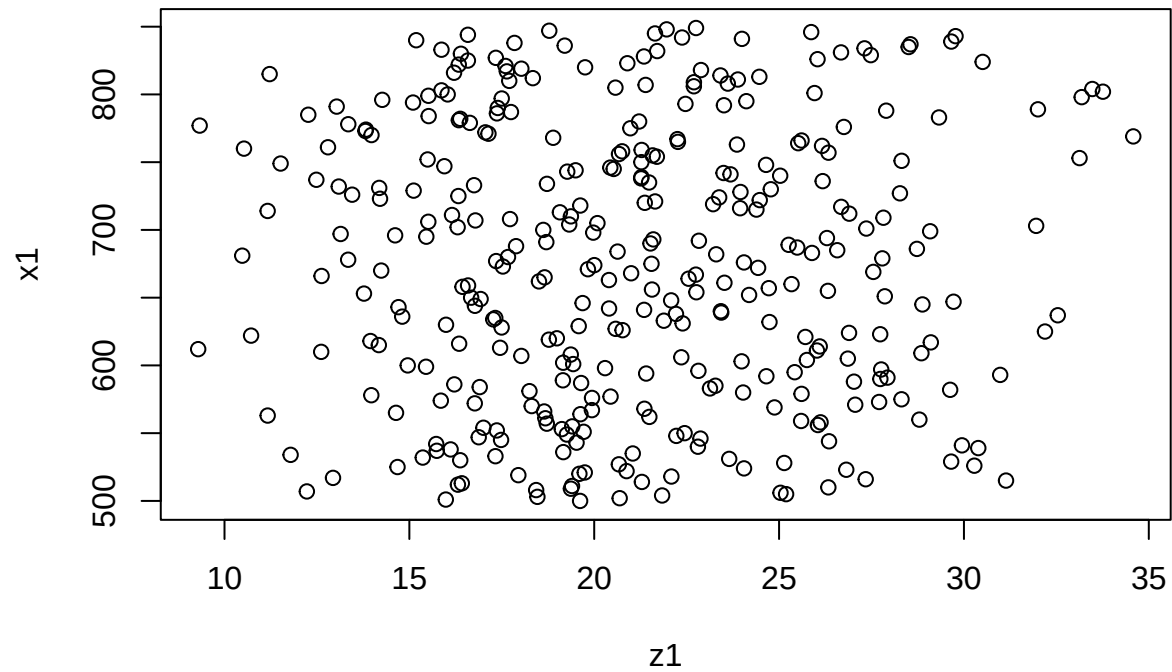
Utilizamos `rnorm` para generar una distribución normal[cite: 1].

```
z1 <- rnorm(350, 21, 5)
plot(z1)
```



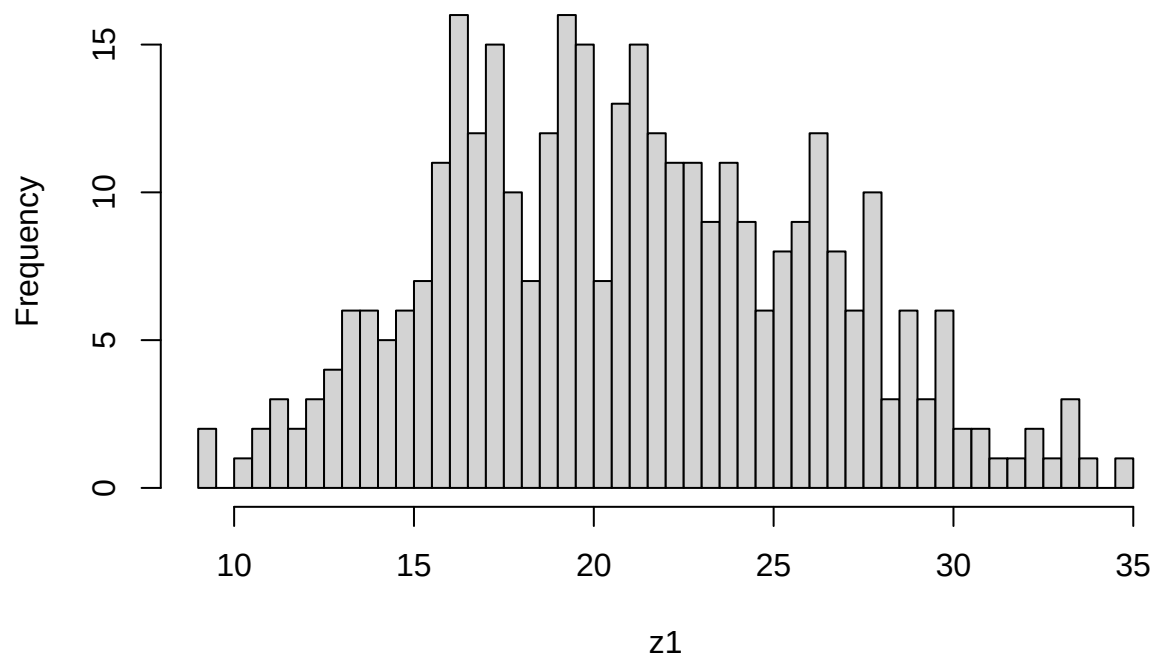
```
w1 <- length(z1)
x1 <- (500:849)
```

```
plot(z1, x1)
```

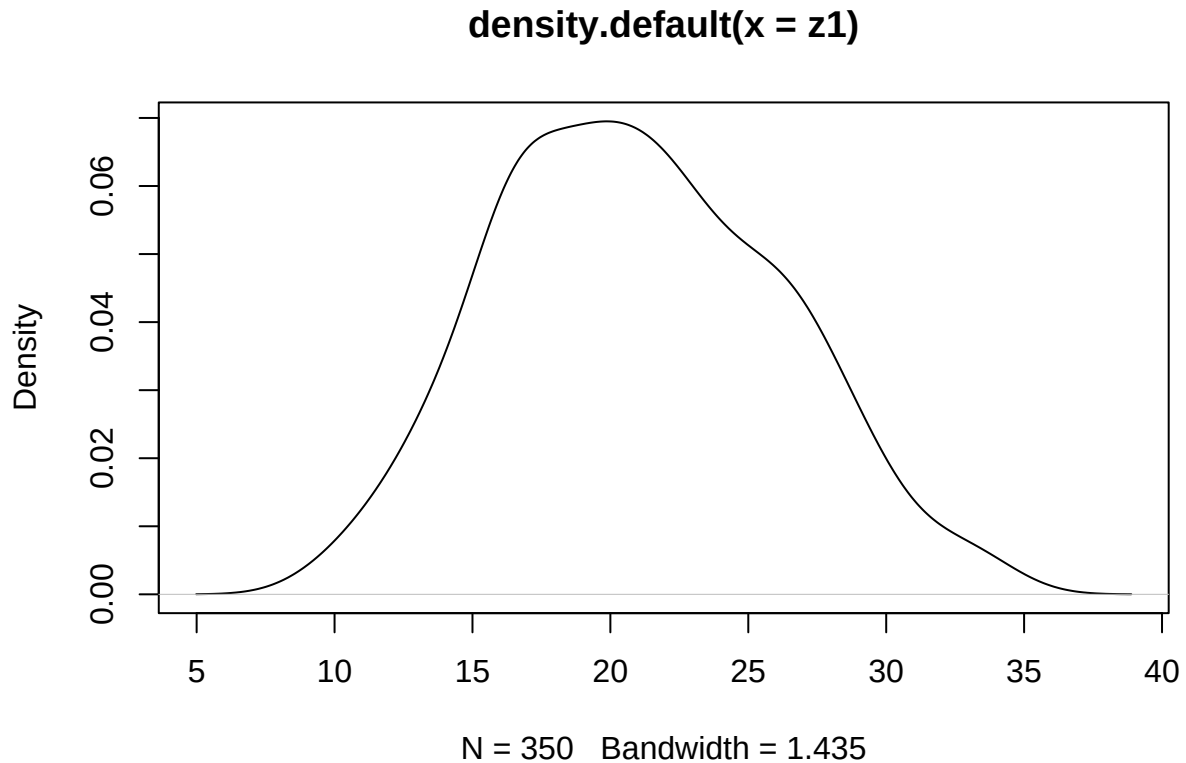


```
hist(z1, main="Histograma de edades", breaks=50)
```

Histograma de edades



```
plot(density(z1))
```



Ejercicio 1 de algoritmos

Consigna: Últimas 3 cifras del DNI (608).

```
DNI=608  
Secuencia_dni <- (1:608)
```

Ejercicio 2: Suma de valores

```
total <- 0  
Valor_final <- length(Secuencia_dni)  
for(i in 1:Valor_final) {  
  total <- total + i  
}  
total
```

```
## [1] 185136
```

Ejercicio 3: Repetir en Python

Nota: Ajusté el rango a 608 para que incluya el último número y coincida con R.

```
suma = 0  
for i in range(1, 608):  
    suma += i  
print(suma)
```

```
## 184528
```

Ejercicio 4: Tiempo de ejecución

```
inicio <- Sys.time()
total <- 0
for(i in 1:length(Secuencia_dni)){
  total <- total + i
}
final <- Sys.time()
final - inicio
```

```
## Time difference of 0.003035307 secs
```

Ejercicios del profe Palma

1.4 Generar un vector secuencia

Comparamos bucle for contra función nativa seq[cite: 1].

```
system.time({
  A <- numeric(50000)
  for (i in 1:50000) { A[i] <- (i * 2) }
})
```

```
##      user  system elapsed
## 0.005    0.000    0.005
```

```
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##      user  system elapsed
## 0.000    0.001    0.001
```

1.5 Serie de Fibonacci

Implementación iterativa vs. recursiva[cite: 1].

```
n <- 30
# Método 1: Bucle for
system.time({
  fib <- numeric(n)
  fib[1] <- 0; fib[2] <- 1
  for (i in 3:n) { fib[i] <- fib[i-1] + fib[i-2] }
})
```

```
##      user  system elapsed
## 0.003    0.000    0.003
```

```
# Método 2: Recursiva
fib_recursiva <- function(k) {
  if (k <= 1) return(k)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}
system.time({ resultado_rec <- fib_recursiva(n) })
```

```
##      user  system elapsed
## 0.913    0.000    0.919
```

1.6 Definición matemática recurrente (Bucle While)

¿Cuántas iteraciones para superar 1.000.000?[cite: 1]

```
fib_w <- c(0, 1)
i <- 2
while (fib_w[i] <= 1000000) {
  i <- i + 1
  fib_w[i] <- fib_w[i-1] + fib_w[i-2]
}
cat("Número:", fib_w[i], "| Iteraciones:", i - 1)
```

```
## Número: 1346269 | Iteraciones: 31
```

1.7 Método burbuja vs Sort

```
set.seed(123)
vec <- sample(1:1000, 1000)

bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n - 1)) {
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        temp <- x[j]; x[j] <- x[j + 1]; x[j + 1] <- temp
      }
    }
  }
  return(x)
}

system.time({ bubble_sort(vec) })
```

```
##      user  system elapsed
##  0.060    0.000    0.061
```

```
system.time({ sort(vec) })
```

```
##      user  system elapsed
##  0.001    0.000    0.000
```

1.8 La penitencia de Newton

Comparación de fuerza bruta contra la fórmula de Gauss[cite: 1, 2].

```
n_newton <- 1e7
# Bucle For
system.time({
  s_iter <- 0
  for (i in 1:n_newton) { s_iter <- s_iter + i }
})

##      user  system elapsed
##  0.159    0.000    0.160

# Gauss
system.time({ s_analitica <- (n_newton * (n_newton + 1)) / 2 })
```

##	user	system	elapsed
##	0	0	0